

Data Structures (CI33/CY33)

Unit 1

POINTERS and ARRAYS:

THE ADDRESS AND INDIRECTION OPERATORS:

To determine the address of a variable, the operator that is available in C is “&” (address of) operator. The operator & and the immediately preceding variable returns the address of the variable associated with it.

C’s other unary pointer operator is the “*”, also called as value at address or indirection operator. It returns a value stored at that address.

Write a program to print the address associated with a variable and value stored at that address.

```
/* Program to print the address associated with a variable and value stored at that address*/  
# include <stdio.h>  
main()  
{  
    int qty = 5;  
    printf("Address of qty = %u\n",&qty);  
    printf("Value of qty = %d \n",qty);  
    printf("Value of qty = %d",*(&qty));  
}
```

OUTPUT

Address of qty = 65524

Value of qty = 5

Value of qty = 5

POINTER TYPE DECLARATION AND ASSIGNMENT

```
ptr = &qty;
```

In C, every variable must be declared for its data type before it is used. Even this holds good for the pointers too. We know that ptr is not an ordinary variable like any integer variable. We declare the data type of the pointer variable as that of the type of the data that will be stored at the address to which it is pointing to. Since ptr is a variable, which contains the address of an integer variable qty, it can be declared as:

```
int *ptr;
```

where ptr is called a pointer variable. In C, we define a pointer variable by preceding its name with an asterisk(*). The “*” informs the compiler that we want a pointer variable, i.e. to set aside the bytes that are required to store the address in memory.

The int says that we intend to use our pointer variable to store the address of an integer. Consider the following memory map:



```

/* Program below demonstrates the relationships we have discussed so far */
# include <stdio.h>
main()
{
    int qty = 5;
    int *ptr; /* declares ptr as a pointer variable that points to an integer variable*/
    ptr = &qty; /* assigning qty's address to ptr -> Pointer Assignment */
    printf("Address of qty = %u \n", &qty);
    printf("Address of qty = %u \n", ptr);
    printf("Address of ptr = %u \n", &ptr);
    printf("Value of ptr = %d \n", ptr);
    printf("Value of qty = %d \n", qty);
    printf("Value of qty = %d \n", *(&qty));
    printf("Value of qty = %d", *ptr);
}

```

OUTPUT

```

Address of qty = 65524
Address of ptr = 65522
Value of ptr = 65524
Value of qty = 5
Value of qty = 5
Value of qty = 5

```

Example :

```

int x; /* x is initialized to a value 10*/
p = &x; /* Pointer declaration & Assignment */
*p=10;

# include <stdio.h>
main()
{
    int *p; /* a pointer to an integer */
}

```

```
int x;  
p = &x;  
*p=10;  
printf("The value of x is %d",*p);  
printf("\nThe address in which the x is stored is %d",p);  
}  
OUTPUT  
The value of x is 10  
The address in which the x is stored is 52004
```

This statement puts the value of 20 at the memory location whose address is the value of px. The value of px is the address of x and so the old value of x is replaced by 20. This is equivalent to assigning 20 to x. Thus we can change the value of a variable indirectly using a pointer and the indirection operator.

Null Pointer Assignment

To assign an integer value to a pointer variable is normally not possible. An exception is an assignment of 0, which is sometimes used to indicate some special condition. A macro is used to represent a null pointer. That macro goes under the name NULL.

Thus, setting the value of a pointer using the NULL, as with an assignment statement such as ptr = NULL, tells that the pointer has become a null pointer.

POINTER ARITHMETIC

Pointer variables can also be used in arithmetic expressions. The following operations can be carried out on pointers:

- Pointers can be incremented or decremented to point to different locations like

```
ptr1 = ptr2 + 3;  
ptr ++;  
-- ptr;
```

However, ptr++ will cause the pointer ptr to point the next address value of its type. For example, if ptr is a pointer to float with an initial value of 65526, then after the operation ptr ++ or ptr = ptr+1, the value of ptr would be 65530. Therefore, if we increment or decrement a pointer, its value is increased or decreased by the length of the data type that it points to.

ARRAYS AND POINTERS

Pointers and arrays are so closely related. An array declaration such as `int arr[ 5 ]` will lead the compiler to pick an address to store a sequence of 5 integers, and `arr` is a name for that address. The array name in this case is the address where the sequence of integers starts. Note that the value is not the first integer in the sequence, nor is it the sequence in its entirety. The value is just an address.

Now, if `arr` is a one-dimensional array, then the address of the first array element can be written as `&arr[0]` or simply `arr`. Moreover, the address of the second array element can be written as `&arr[1]` or simply `(arr+1)`. In general, address of array element `(i+1)` can be expressed as either `&arr[ i ]` or as `(arr+ i)`.

Thus, we have two different ways for writing the address of an array element. In the latter case i.e, expression `(arr+ i)` is a symbolic representation for an address rather than an arithmetic expression. Since `&arr[ i ]` and `(ar+ i)` both represent the address of the `i`th element of `arr`, so `arr[ i ]` and `*(ar + i)` both represent the contents of that address i.e., the value of `i`th element of `arr`.

Note that it is not possible to assign an arbitrary address to an array name or to an array element. Thus, expressions such as `arr`, `(arr+ i)` and `arr[ i ]` cannot appear on the left side of an assignment statement. Thus we cannot write a statement such as:

```
&arr[0] = &arr[1]; /* Invalid */
```

However, we can assign the value of one array element to another through a pointer, for example,

```
ptr = &arr[0]; /* ptr is a pointer to arr[ 0 ] */
```

```
arr[1] = *ptr; /* Assigning the value stored at address to arr[1] */
```

```
/* Program that accesses array elements of a one-dimensional array using pointers */
# include<stdio.h>
main()
{
    int arr[ 5 ] = {10, 20, 30, 40, 50};
    int i;
    for(i = 0; i < 5; i++)
    {
        printf("i=%d\t arr[i]=%d\t *(arr+i)=%d\t", i, arr[i], *(arr+i));
        printf("&arr[i]=%u\t arr+i=%u\n", &arr[i],(arr+i));
    }
}
```

OUTPUT:

```
i=0 arr[i]=10 *(arr+i)=10 &arr[i]=65516 arr+i=65516
i=1 arr[i]=20 *(arr+i)=20 &arr[i]=65518 arr+i=65518
```

```

i=2 arr[i]=30 *(arr+i)=30 &arr[i]=65520 arr+i=65520
i=3 arr[i]=40 *(arr+i)=40 &arr[i]=65522 arr+i=65522
i=4 arr[i]=50 *(arr+i)=50 &arr[i]=65524 arr+i=65524

```

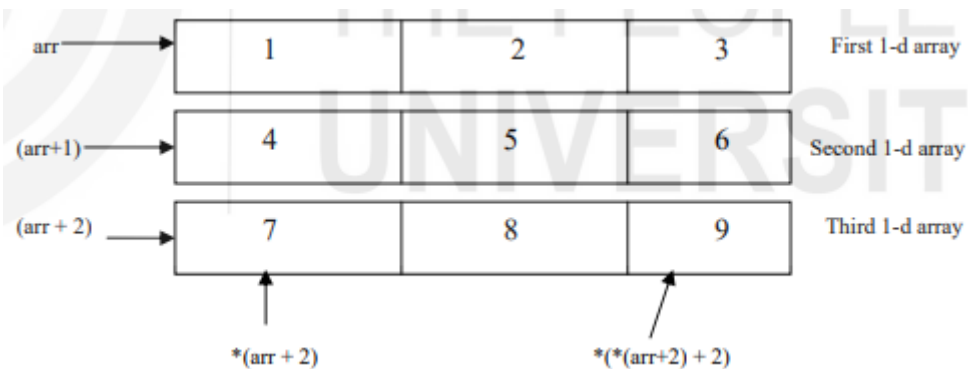
Note that i is added to a pointer value (address) pointing to integer data type (i.e., the array name) the result is the pointer is increased by i times the size (in bytes) of integer data type. Observe the addresses 65516, 65518 and so on. So if ptr is a char pointer, containing addresses a , then $ptr+1$ is $a+1$. If ptr is a float pointer, then $ptr+1$ is $a+4$.

POINTERS AND MULTIDIMENSIONAL ARRAYS

C allows multidimensional arrays, lays them out in memory as contiguous locations, and does more behind the scenes address arithmetic. Consider a 2-dimensional array.

```
int arr[ 3 ][ 3 ] = { {1, 2, 3}, {4, 5, 6}, {7, 8, 9} };
```

The compiler treats a 2 dimensional array as an array of arrays. As you know, an array name is a pointer to the first element within the array. So, arr points to the first 3-element array, which is actually the first row (i.e., row 0) of the two-dimensional array. Similarly, $(arr + 1)$ points to the second 3-element array (i.e., row 1) and so on. The value of this pointer, $*(arr + 1)$, refers to the entire row. Since row 1 is a onedimensional array, $(arr + 1)$ is actually a pointer to the first element in row 1. Now add 2 to this pointer. Hence, $*(arr + 1) + 2$ is a pointer to element 2 (i.e., the third element) in row 1. The value of this pointer, $*(*(arr + 1) + 2)$, refers to the element in column 2 of row 1. These relationships are illustrated below:



POINTERS AND STRINGS

As we have seen in strings, a string in C is an array of characters ending in the null character (written as `'\0'`), which specifies where the string terminates in memory.

Like in one-dimensional arrays, a string can be accessed via a pointer to the first character in the string. The value of a string is the (constant) address of its first character. Thus, it is appropriate to say that a string is a constant pointer.

A string can be declared as a character array or a variable of type `char *`. The declarations can be done as shown below:

```
char country[ ] = "INDIA";  
char *country = "INDIA";
```

Each initialize a variable to the string "INDIA". The second declaration creates a pointer variable country that points to the letter I in the string "INDIA" somewhere in memory.

Once the base address is obtained in the pointer variable country, *country would yield the value at this address, which gets printed through,

```
printf("%s", *country);
```

DYNAMIC MEMORY ALLOCATION IN C

Dynamic memory allocation in C, a language, refers to the process of allocating memory during program execution (runtime) rather than at compile time. This capability is crucial for handling situations where the required memory size is unknown in advance, such as when dealing with user input or variable-sized data structures.

C provides a set of standard library functions, primarily found in <stdlib.h>, for managing dynamic memory on the heap:

malloc(): Allocates a block of memory of a specified size in bytes and returns a pointer to the beginning of the allocated block. If allocation fails, it returns NULL.

```
int *ptr = (int *)malloc(5 * sizeof(int)); // Allocates space for 5 integers
```

calloc(): Allocates a block of memory for an array of a specified number of elements, each of a specified size, and initializes all allocated bytes to zero. It also returns a pointer to the beginning of the block.

```
int *arr = (int *) calloc (10, sizeof(int)); //Allocates space for 10 integers, initialized to 0
```

realloc (): Resizes a previously allocated block of memory. It can either expand or shrink the block and returns a pointer to the new memory block.

```
ptr = (int *) realloc (ptr, 10 * sizeof(int)); // Resizes the block pointed to by ptr
```

free (): Deallocates the memory block previously allocated by malloc (), calloc (), or realloc (), making it available for reuse.

```
free(ptr); // Releases the memory pointed to by ptr
```

MALLOC AND CALLOC

The main differences are that malloc () allocates a single block of memory and does not initialize it, while calloc() allocates multiple memory blocks and initializes them to zero. malloc () takes a single argument (the total number of bytes), whereas calloc() takes two arguments: the number of elements and the size of each element.

Feature	malloc()	calloc()
Memory Initialization	The memory block contains garbage values.	The memory block is initialized to all zeros.
Number of Arguments	One argument: the size of the memory block in bytes.	Two arguments: the number of elements and the size of each element.
Memory Allocation	Allocates a single, contiguous block of memory.	Allocates multiple, contiguous blocks of memory.
Execution Speed	Faster because it doesn't perform the initialization step.	Slower because it includes the time to initialize the memory to zero.

C Example to swap two numbers using pointers

```

* Program to swap two numbers using pointers*/
#include <stdio.h>

// function to swap the two numbers
void swap (int *x, int *y)
{
    int t;
    t    = *x;
    *x    = *y;
    *y    = t;
}

int main ()
{
    int num1, num2;

    printf("Enter value of num1: ");
    scanf("%d",&num1);
    printf("Enter value of num2: ");
    scanf("%d",&num2);

    //displaying numbers before swapping
    printf("Before Swapping: num1 is: %d, num2 is: %d\n",num1,num2);
    //calling the user defined function swap()
    swap(&num1,&num2);
    //displaying numbers after swapping
    printf("After Swapping: num1 is: %d, num2 is: %d\n",num1,num2);
    return 0;
}

```

C Example to find sum of 5 elements in an array using pointers

```
#include <stdio.h>

int main() {
    int arr[5];    // Declare an array of integers
    int *ptr;      // Declare an integer pointer
    int sum = 0;   // Initialize sum
    int i;

    // Input array elements
    printf("Enter 5 elements of the array: ");
    for(i = 0; i < 5; i++) {
        scanf("%d", &arr[i]);
    }

    // Use the pointer to traverse and compute the sum
    ptr = arr;    // Point to the first element of the array
    for(i = 0; i < 5; i++) {
        sum += *ptr; // Add the value pointed by ptr to sum
        ptr++;      // Move the pointer to the next array element
    }

    // Display the result
    printf("Sum of array elements: %d\n", sum);

    return 0;
}
```

Sum of all elements in array using malloc() function

```
#include <stdio.h>
#include <malloc.h>

void main()
{
    int i, n, sum = 0;
    int *a;

    printf("Enter the size of array A \n");
    scanf("%d", &n);

    a = (int *) malloc(n * sizeof(int));

    printf("Enter Elements of the List \n");
    for (i = 0; i < n; i++)
    {
        scanf("%d", a + i);
    }
}
```



```

    }

    /* Compute the sum of all elements in the given array */

    for (i = 0; i < n; i++)
    {
        sum = sum + *(a + i);    /* this *(a+i) is used to access the value stored at the address*/
    }

    printf("Sum of all elements in array = %d\n", sum);
    return 0;
}

```

CLASSIFICATION OF DATA STRUCTURES

